AgentUCT: Budgeted Workflow Search with Reusable Execution

Mathematical Proofs and Formal Verification

This proof reader follows the canonical proof sources. Its equations, tables and sections have independent numbering. An **M.** prefix identifies a main-paper item; an **S.** prefix identifies an item in the independently numbered supplementary material. Full machine-checked proof bodies are in `theory/lean/AgentUCTCompleteProofsEN.lean`.

1 Operational Execution and Backup Identities

The formal model represents a completed evaluation by an expanded path v_t, a terminal completion x_t extending it, and its reward. The legal-tree recurrence starts with the root. An event either evaluates an already expanded terminal or inserts one new child of an existing node. Inserting a child preserves prefix closure. The allocation predicate additionally requires unvisited children first and index maximality whenever all legal children have positive counts. Legality alone is not a definition of UCT.

1.1 Counts, Returns, and Birth Events

Inducting over the event list in (M.4) gives

$$N_t(p) = \sum_{s \le t} \mathbf{1}\{p \preceq v_s\}, \tag{1}$$

$$S_t(p) = \sum_{s \le t} \mathbf{1}\{p \preceq v_s\} f(x_s). \tag{2}$$

Every backed-up terminal extends p, so $0 \le Q_t(p) \le V(p) \le 1$ for positive counts. Subtracting the return sum from $N_t(p)V(p)$ proves (M.19) and $Q_t(p) = V(p) - D_t(p)/N_t(p)$. At a terminal, prefix-antichain legality forces all its completed returns to equal its fixed value. Hence its deficit is zero.

For internal p, partition backed-up events by $v_s = p$ or by the first action a after p in v_s. The former can occur only once, when a non-root internal node is inserted. The latter are exactly the child visits. This proves (M.20), with a birth deficit between zero and one. The local action stream at p contains only the latter events. If it has length n, its child counts are exactly the global backed-up child counts, while the parent count is n at the root and $n + 1$ at an already born non-root internal node. Projecting a global history onto this stream therefore preserves the index with a fixed offset $o \in \{0, 1\}$. This step accounts for progressive expansion rather than treating every internal node as a stationary bandit.

1.2 Query-Specific Execution and Conservation

For each query and terminal, saved depths form an initial segment under prefix completeness. Its largest saved depth exists because the path is finite. A demanded key is absent precisely when its depth exceeds that maximum. Summing its nonnegative weight proves equivalence between the set-difference bill and deepest-checkpoint suffix execution. Publishing a complete path preserves prefix completeness.

For one successful evaluation, disjointness of the old ledger and the newly demanded keys gives

$$\sum_{r \in M_{t+1}} w_r - \sum_{r \in M_t} w_r = \sum_{r \in \mathcal{D}(x_t) \setminus M_t} w_r.$$

The current and historical graphs remain equal because both receive the same union update from equal initial states. Telescoping proves (S.16). Nonreusable overhead adds $\sum_t h_t(x_t)$ and never enters this finite resource budget. The proof applies to the same events used for reward backup, independently of how they were selected. Distinct query identifiers make their resource sets disjoint.

For (S.5), apply the union accounting identity first to K, A and then to $K \cup A, B$, and cancel the intermediate cache weight. This allows A and B to overlap. The operational transition publishes $\mathcal{D}(x_t)$ regardless of the depth of v_t; a prefix of x_t outside the expanded ancestor chain enters the cache and leaves its tree statistics unchanged. The formal statement is `rollout_publication_without_backup`.

2 Explicit Proof of the Tree Regret Bound

This section gives the constants used by the Lean main theorem. Let K_p be the number of legal children of p, $K = \max_p K_p$, and $D = \max_{x \in \mathcal{X}} |x|$. Choose $\delta > 0$ no larger than every positive gap Δ_{pa}. Finiteness supplies such a number; when every gap is zero, any positive δ suffices. This choice is made before quantifying over histories or horizons.

The penalty in (M.6) is bounded by

$$H = \gamma \sum_{r \in \mathcal{D}(X)} M_r.$$

For a target child count $m \in \mathbb{N}$, define

$$L_p(m) = \left\lceil \max\left\{ K_p + 1, \exp\left(\frac{(H+2)^2 m + 1}{c^2} \right), (8K_p \max\{1, c^2\} + 2)^2 \right\} \right\rceil, \tag{3}$$

$$F_p(m) = 2L_p(m).$$

Lemma 1 (Finite local exploration window)**.** *At an internal node, suppose means lie in $[0,1]$, penalties lie in $[0,H]$, unvisited children are taken first, and subsequent choices maximize the actual index with $c > 0$ and parent offset $o \in \{0,1\}$. After $F_p(m)$ local child choices, every child has more than m visits. Means and penalties may otherwise vary arbitrarily with the history.*

Proof. Initialization visits each child within the first K_p choices. Let $L = L_p(m)$ and suppose some child has at most m visits at local time $2L$. Throughout the interval $[L, 2L)$, its count is positive and at most m. The exponential term in (3) gives

$$c^2 \log(L + o + 1) > (H+2)^2 m.$$

Its exploration bonus therefore exceeds $H + 2$. Its index exceeds two, since its mean is nonnegative and its penalty is at most H. A selected child's mean is at most one and its penalty is nonnegative, so its bonus must exceed one. Thus each selected child has prior count at most $c^2 \log(2L + o + 1)$. Each of the K_p children can be selected at such counts at most $c^2 \log(2L + o + 1) + 1$ times.

The square term in (3) ensures

$$K_p\left(c^2 \log(2L + o + 1) + 1\right) < L.$$

For completeness, $\log z \le 2\sqrt{z}$ for $z > 0$, $2L + o + 1 \le 4L$, and $\sqrt{L} \ge 8K_p \max\{1, c^2\} + 2$ imply this inequality. There are L choices in the interval, contradicting the count bound. All arguments concern a finite observed history; no infinite continuation is assumed. □

2.1 Coverage and Disappearance of Cold Penalties

Define

$$C_0 = 1, \qquad C_{d+1} = 1 + \max_{p \in \mathcal{T}} F_p(C_d), \qquad T_{\text{cache}} = C_D. \tag{4}$$

We prove by remaining depth that C_d visits to a node suffice for every terminal below it to be visited. At depth zero, a positive visit evaluates that terminal. At an internal node, discarding its possible single birth event leaves at least $F_p(C_d)$ local choices. Lemma 1 gives each child more than C_d visits, and the induction hypothesis applies. The root count is the number of completed evaluations, so every terminal has been visited once $T \ge T_{\text{cache}}$.

Each such visit materializes all resource pairs for its terminal and complete panel. Therefore K_T covers $\mathcal{D}(\mathcal{X})$ after T_{cache}, and every cold forecast becomes zero. Before that time, normalization and nonnegative forecast weights bound it by H. This derives both the bound and disappearance from the execution model; neither is postulated as a property of an unknown error process.

2.2 Closing the Deficit Induction

Set $A_0 = B_0 = 0$. Given A_d, B_d, define

$$\begin{aligned} m_d &= \left\lceil \max\left\{1, (32A_d/\delta)^2, 8B_d/\delta\right\}\right\rceil, \\ \tau_d &= \max_{p\in\mathcal{T}} \max\{T_{\text{cache}}, F_p(m_d)\}, \\ A_{d+1} &= K\left(A_d + \frac{4c^2}{\delta^2}\right), \\ B_{d+1} &= 1 + K(B_d + \tau_d + 1). \end{aligned} \tag{5}$$

We prove simultaneously for every finite legal event history and every node of remaining depth at most d that

$$D_t(p) \le A_d \log(N_t(p) + 1) + B_d. \tag{6}$$

The depth-zero statement follows from the exact terminal mean. Assume the result for all child subtrees and all history prefixes. For $n \ge m_d$, the inequalities $\log(n+1) \le 4\sqrt{n}$ and (5) give

$$\frac{A_d \log(n+1) + B_d}{n} \le \delta/4.$$

After τ_d local choices at an internal node, every child exceeds this count threshold by Lemma 1. Global time is at least local time, so the cold penalties have also disappeared. An optimal child a^* has

$$Q_t(pa^*) \ge V(p) - \delta/4, \qquad Q_t(pa) \le V(pa)$$

for any child a. If a child with positive gap is selected, index maximality and the nonnegative optimal-child bonus imply that its own bonus is at least $\delta/2$. Consequently its prior count is at most

$$\frac{4c^2 \log(N_t(p) + 1)}{\delta^2}.$$

Monotonicity of counts and of the logarithm bounds all late selections by the final parent count. Counting the at most τ_d early selections and the last late increment yields

$$N_T(pa) \le \tau_d + \frac{4c^2 \log(N_T(p) + 1)}{\delta^2} + 1 \tag{7}$$

for each positive-gap child. Zero-gap children contribute nothing to $N_T(pa)\Delta_{pa}$.

Insert (7) and the child induction hypotheses into (M.20). Use $0 \le \Delta_{pa} \le 1$, $N_T(pa) \le N_T(p)$, the birth bound one, and at most K children. This gives exactly (6) with the constants in (5). At the root, $N_T = T$ and $D_T = R_Q(T)$, proving Theorem M.2. All constants were fixed before the arbitrary history and are independent of T.

Ordinary UCT uses the same induction with $H = 0$ and $T_{\text{cache}} = 0$. Proposition S.2 transfers that bound to the legacy direct-edge estimator without adding a cache-coverage delay: initialized direct edges are already published. The current first-unpaid estimator instead uses its domination by the complete cold bill and the coverage argument. For the full-cold policy the explicit coverage constants can be enormous, particularly at greater depth. The proof establishes a finite instance-dependent bound, not efficient worst-case discovery.

2.3 Reading the Lean Proof in Dependency Order

The following steps identify the actual proof obligations. The English guide `theory/lean/AgentUCTProofGuideEN.lean` follows the same order; its complete-proof companion `theory/lean/AgentUCTCompleteProofsEN.lean` contains all definitions and proof bodies in dependency order, with English step comments and source-module locations. Each `#check` prints the full declaration and its explicit hypotheses; it does not add a theorem.

1. The backup recurrence gives counts and return sums. `operational_mean_deficit` converts the actual mean to $V - D/N$ for positive N. The parent partition retains the single birth deficit and the local-stream offset; these are operational identities.

2. Nonnegative normalized completion laws and a fixed price envelope give a history-independent penalty bound. It constrains predictions, not physical receipts or admission of model calls.

3. `finite_window_explores` proves the finite-window contradiction. `finite_explicit_window_explores` supplies the explicit exponential and square thresholds. `actual_children_explored` transfers the statement to the actual local action stream.

4. `actual_subtree_coverage` inducts on remaining depth, retaining the possible birth visit. `actual_restorable_coverage` converts complete terminal publications to persistent executable resources.

5. `coldForecastPenalty_after_coverage` derives zero penalties from empty missing-resource sets. Bounded penalties alone establish exploration, and do not discharge this obligation.

6. `logarithmic_deficit_small` divides the child induction bound by its positive count to obtain $D/N \le \delta/4$. The optimal child then has mean at least $V(p) - \delta/4$.

7. `actual_suboptimal_child_count` combines this mean bound, zero penalties, index maximality and the bonus-square identity. It retains the early threshold and the last increment.

8. `parent_deficit_log_bound` sums the parent decomposition; `actual_subtree_log_deficit` closes the remaining-depth induction simultaneously for every history prefix. Root count and deficit identities give the cumulative training-quality bound.

9. `onlineMean_envelope` bounds the online price mean. `sampledLaw_mass` retains multiplicity and requires a positive sample count. They discharge estimator premises of `online_mean_shared_root_log_bound`; fixed scores and successful persistent publication remain execution assumptions.

Table 1: Proof-to-claim map. The local recurring-cost counterexample and conditional confidence results retain their own hypotheses; they are not the proof of the operational tree rate.

Paper result	Representative Lean declaration in namespace `AgentUCTPaper`
Full-panel compression	`full_panel_residual_quotient`, `full_panel_history_rectangular`
Demand-profile compression	`demand_group_cost_quotient`, `demand_group_publication`, `demand_separating_merge_loses_information`
Query-selective information loss	`GraphWitness.no_count_only_optimal_policy`, `GraphWitness.budget_feasibility_reverses`
Actual reward backup	`replayBackup_visits`, `replayBackup_mean`, `operational_mean_deficit`
Graph accounting and suffix	`replayMetered_cold_conservation`, `coldWork_eq_deepest_suffix`
Tree/stream correspondence	`localBefore_parent_count`, `projected_index_max`
Exploration and coverage	`actual_children_explored`, `actual_restorable_coverage`
Online-price tree rate	`checkpoint_conditioned_shared_root_log_bound`, `online_mean_shared_root_log_bound`
Fixed-price special case	`cold_forecast_uniform_root_log_bound`
Shared fixed-panel family	`shared_cache_uniform_root_log_bound`, `fixed_panel_family_log_bound`
Completion index and budget	`completion_index_error`, `own_cache_budget_coverage`
Same-state pricing benefit	`priced_choice_tradeoff`, `physical_saving_certificate`, `actual_net_gain_certificate`
Exclusive trajectory costs	`own_cache_cost_difference`
Own-cache cumulative objective	`own_cache_policy_regret_bound`, `own_cache_cost_difference_common_overhead`
Recommendation and endpoint	`validation_shortlist_gap`, `equal_quality_positive_endpoint_gap`
Full cost and its errors	`referenceColdCost_resource_form`, `full_reference_cost_error`
Prediction and optimism	`full_cost_mean_unique_mse_minimizer`, `selected_net_gap_from_cost_radii`
Legacy direct-edge reduction	`next_edge_allocation_iff_uct`, `next_edge_uniform_root_log_bound`, `legacyContinuationCold_common_shift`
First unpaid execution	`nextUnpaidBill_le_complete`, `dominated_cold_shared_root_log_bound`
Coupled trajectories	`resolvedTreeHistory_valid`, `next_edge_coupled_history_eq_uct`, `next_edge_coupled_state_eq_uct`
Local recurring-cost failure	`next_edge_linear_regret`

3 Other Objectives and Estimator Proofs

3.1 Independent Cache Trajectories and Endpoints

Each comparator reward is at most V^*, so its cumulative reward advantage is at most $R_Q(T)$. The actual cold bill is at most W and the comparator cold bill is nonnegative, even though they use different caches. This proves (S.18). Equal recurring-overhead totals cancel before applying the cold-budget bound.

The best observed training reward is at least the average observed reward. Subtracting from V^* gives the simple-gap bound. After terminal coverage, an observed training maximizer is a global table maximizer. For (S.19), let x_g maximize validation and let x_f be the training-ranked head. The head belongs to every nonempty top-k shortlist, so $g(x_{\text{end}}) \geq g(x_f)$. Therefore

$$\begin{aligned} g(x_g) - g(x_{\text{end}}) &\leq f(x_g) + \epsilon - f(x_f) + \epsilon \\ &\leq R_Q(T)/T + 2\epsilon. \end{aligned}$$

The uniform table difference is a hypothesis. For a counterexample without it, two fully observed workflows with training scores $(1, 0)$ and validation scores $(0, 1)$ give validation regret one after training top-1 selection.

For endpoint utility, direct subtraction gives

$$J^\pi - J = \big(q_{\text{rep}}(x^\pi_{\text{end}}) - q_{\text{rep}}(x_{\text{end}})\big) + \frac{\lambda_J}{C_{\text{ref}}}(C_{\text{opt}} - C^\pi_{\text{opt}}).$$

Equal quality and a strictly larger actual bill give a strictly positive gap when $\lambda_J, C_{\text{ref}} > 0$. Cumulative regret and endpoint utility thus require distinct interpretations.

3.2 Cost Targets and Error Bounds

For the resource formula (M.12), exchange the two finite sums in (M.6). The coefficient of each forecast weight is the reference probability that this resource is demanded and absent from the current graph. Shared-resource dependence never enters this operation.

To isolate the effect of publication, partition $\mathcal{D}(x) \setminus K$ into the disjoint sets $\mathcal{D}(x) \setminus (K \cup A)$ and $(\mathcal{D}(x) \cap A) \setminus K$. Subtracting the first bill leaves exactly the second. Averaging with the same law and weights proves (S.6). This argument formalizes sequential cache accumulation and its effect on prediction without treating a prediction as physical execution; see `coldWork_sequential_union`, `referenceColdCost_publication_savings`, and `referenceColdCost_cache_antitone`.

For (S.1), expand the square around the full conditional mean m_t. The cross term is zero because $\mathbb{E}[b_t(X) - m_t] = 0$. The remaining excess is $(z - m_t)^2$, which vanishes exactly at $z = m_t$. The formal finite-sum theorem needs normalization; nonnegativity makes the normalized weights a probability law.

To prove Proposition S.1, insert $\sum_x \rho_C(x) b_t(x)$ between the forecast mean and the actual mean. The first difference is bounded by the reference-weighted bill errors. Apply the triangle inequality within each cold resource sum and to the overhead. Bound the second difference by $B_{\max} \sum_x |\rho_C(x) - \rho_{\text{exec}}(x)|$. The estimate explicitly includes both sources of error.

Let $|\widehat{b} - b| \leq e$ and $b \geq 0$. Then $0 \leq \underline{b} = \max\{0, \widehat{b} - e\} \leq b$ and $b - \underline{b} \leq 2e$. These facts prove (S.9). If a maximizes that optimistic index and a^* maximizes true net value, then

$$\begin{aligned} \mu_{a^*} - \gamma b_{a^*} &\leq Q_{a^*} + r_{a^*} - \gamma \underline{b}_{a^*} \\ &\leq Q_a + r_a - \gamma \underline{b}_a \\ &\leq \mu_a + 2r_a - \gamma b_a + 2\gamma e_a. \end{aligned}$$

The last step uses the selected arm's reward upper-error bound. This proves the stated conditional net-gap inequality without asserting that tree means satisfy those reward confidence conditions.

3.3 Reduction and Recurring-Cost Counterexample

A positive child count witnesses an earlier complete event whose expanded path passes through that child. Its terminal completion extends the child and has materialized the child's entire panel demand. Persistent publication retains that demand, so its current cold bill is zero for every nonnegative predictor. The same holds for its parent. Equation (S.7) is zero, and the actual initialized-child index comparisons become exactly ordinary UCT comparisons. Unvisited-first clauses are identical as well. This proves equivalence of the whole allocation predicates on a given history.

For trajectory equality, let $\mathcal{E}_{\text{next}}(h)$ and $\mathcal{E}_{\text{UCT}}(h)$ be the complete events that can extend history h while satisfying the same fixed reward table, legal expansion protocol, and respective allocation rule. The predicate equivalence gives $\mathcal{E}_{\text{next}}(h) = \mathcal{E}_{\text{UCT}}(h)$. Use the same initial tree and cache and a shared resolver $\Phi(h, \mathcal{E})$ that returns an admissible event or stops. The resolver fixes expansion, rollout completion, tie-breaking, evaluation outcomes including receipts, and stopping; a common random tape can be fixed as part of it. Equal histories give equal event sets and the same output of Φ. Induction therefore gives equal event histories at every step, hence equal reward backups, expanded trees, and cache graphs. The Lean development proves both resolver validity and this coupled equality. Rollout coupling is necessary: expanding the same node can lead to different leaves and rewards when its completion choices differ.

For historical continuation costs, each terminal extending p demands a superset of p's resources. Subtracting the same cached set preserves inclusion, and nonnegative weights give $\widehat{d}_t(x) \geq \widehat{d}_t(p)$. Remove the inactive clipping and use $\sum_x \rho_C(x) = 1$ to pull the parent term outside the expectation. This proves the common shift. With a common omitted tail the same algebra applies; action-dependent tails introduce the exact pairwise difference given in the main text.

For (S.8), apply the ordinary-UCT quality bound to the next-edge allocation predicate. The comparator's reward at every step is at most V^*. Each trajectory is charged against its own cache, and cancellation of equal recurring-overhead totals leaves a cost difference at most W. Multiplying by $\eta \geq 0$ proves the bound. The complete composed statement is `next_edge_own_cache_log_bound`; it quantifies the gap and constants before all actual and comparator histories. For the local approximation relation, full net gap equals next-edge net gap plus η times chosen-tail minus comparator-tail at each step. Sum these identities and use the assumed tail-variation bounds, as in `next_edge_tail_regret_transfer`. A logarithmic local proxy bound remains a separate premise.

For the recurring-cost example, initialize arm A then arm B. Their immediate-cost net values are 0.4 and 0.6, while their full-cost net values are 0.4 and 0.3. The next-edge UCB rule therefore treats A as the inferior proxy arm with gap 0.2. At any subsequent selection of A, exact deterministic reward means and index maximality imply $0.2 \leq \sqrt{2\log(t+1)/N_A(t)}$. Counting such selections gives a logarithmic upper bound on $N_A(T)$. In particular the conservative bound $N_A(T) \leq 400 \log T + 4$ holds for $T \geq 1$; for $T \geq 2$ the direct count bound is already stronger. Since full-cost regret is $0.1(T - N_A(T))$, the claimed lower bound follows. The preserved Lean proof obtains the same conservative constants via its general local-regret counting lemma.

3.4 Cache Compression and Information Loss

For Proposition M.1, exchange the finite sums over qids and demanded prefixes. This gives the residual bill in (M.8). Full-panel publication adds every qid incidence for each demanded prefix, so exactly those residual bills become zero. The product identity $(Q \times S) \cup (Q \times P) = Q \times (S \cup P)$ proves rectangularity by induction over the publication history. Lean checks these statements in `full_panel_residual_quotient`, `full_panel_residual_update`, and `full_panel_history_rectangular`.

For Proposition S.3, enumerate the two resource sets in the stated witness. Both hit-count vectors and both full-panel residual-bill vectors coincide. The partial-panel bills are $(0, 1)$ and $(1, 0)$. A deterministic function of the identical summary chooses the same action in both states, and therefore loses one unit in one of them. If it chooses u with probability p, expected excess costs are $1 - p$ and p, whose maximum is at least $1/2$. A common estimate z of u's bill has errors $|z|$ and $|z - 1|$, again with maximum at least $1/2$. Finally, the unit recurring bill makes total costs $(1, 2)$ and $(2, 1)$, proving the budget claim. The 12 checked declarations and assumption review are recorded in `theory/evidence/graph-information/REVIEW.txt`. These are information and one-decision results; no multi-round policy-regret lower bound is inferred.

3.5 Completion Decisions and Shared Panels

For Theorem M.1, each allowed panel is a union of profile classes. Exchange the finite sums over queries, groups, and prefixes to obtain Equation (M.7). Publication includes either every member of a class at a prefix or none, so the displayed zeroing update is closed on the summary. For necessity within partition-based residual sums, let a merged class contain $q \in Q_j$ and $r \notin Q_j$. With unit prices, compare a residual vector missing only q to one missing only r. Every group total is identical; the panel bills are one and zero. No decoder of those totals recovers both. One-step prefixes make both caches prefix-complete. The Lean module `TreeDemandQuotient` proves the finite-sum identity, publication rule, profile criterion, and decoder impossibility separately. The necessity statement does not rule out stronger encodings or compressions specific to a restricted set of reachable histories.

For Proposition M.2, unpriced maximality gives $U_b - U_a \geq 0$ and priced maximality gives $U_b - U_a \leq \gamma(\widehat{b}_b - \widehat{b}_a)$. Positive γ implies $\widehat{b}_a \leq \widehat{b}_b$. The triangle inequality gives $b_b - b_a \geq \widehat{b}_b - \widehat{b}_a - \epsilon_a - \epsilon_b$. Applying the reward-error bounds as well proves (S.12). For (S.14), apply sequential cache accounting to $A \cup B$ in both orders and subtract the two expressions. All these arguments are checked in `TreeDecisionValue`. Its numerical example checks only the two displayed one-decision cases, with no hidden claim of a legal full-tree counterexample or of calibrated UCT confidence bounds.

Equation (M.14) is a rearrangement of the two indices. For Proposition S.5, insert $\widehat{b}_a$ and $\widehat{b}_{a^*}$. Estimated-index maximality removes the middle difference; the remaining terms are bounded by $\gamma\epsilon_a$ and $\gamma\epsilon_{a^*}$. Lean also checks the finite-sum version in `completion_index_cumulative_error`. The next-edge reversal follows by subtracting the common edge bill. The scaled information loss follows from $\max\{H(1-p), Hp\} \geq H/2$. Proposition S.6 sums each policy's own costs, with nonnegative prefix costs ensuring every earlier prefix also fits. These declarations are in `TreeFiniteBudget`.

For Theorem S.1, write a local cache as $K_t^{\text{own}} \cup E_t$, where E_t contains publications made outside the local tree. The cold penalty is nonnegative and at most H_j because the completion law is normalized and all resource predictions are nonnegative. Construct the same bounded tree run as in the single-panel proof. At its finite coverage threshold, local publications alone contain every required resource. Their inclusion in $K_t^{\text{own}} \cup E_t$ zeros the cold penalty. The original subtree induction then applies without alteration. The external set may depend on the local history; no independence assumption is used. Summing the resulting local bounds and using $T_j \le T$ gives the family bound, with nonnegative A_j. `TreeSharedCache` checks both statements.

The first unpaid estimator uses an ordered resource list for each query. `nextUnpaidBill` skips cached resources and nonpositive predicted prices; otherwise it returns the current price. Induction gives a nonnegative value bounded by `orderedColdBill` for nonnegative predictions. Summing over queries and averaging under the same nonnegative completion probabilities preserves this inequality. Full coverage zeros the estimate. The theorem `dominated_cold_shared_root_log_bound` proves the whole-tree bound for any nonnegative penalty bounded above by the complete cold forecast. Its proof replaces the full-forecast bound by transitivity and uses that domination to zero the penalty at coverage. The immediate-child reduction is not used. `nextUnpaidBill_cached_edge_counterexample` checks the saved-edge example in the cost-estimator analysis.

3.6 Reachable Separation and Generated Workflows

For Proposition S.4, both demand lists contain four unit-price resources and start from the empty set. Direct set subtraction gives one residual unit at each prefix over the three-query universe. Restricting the next demand to A gives $(0, 1)$ and $(1, 0)$. Adding the common unit cost proves the budget statement. `ReachableDemandSeparation` checks reachability, warm-up bills, residual equality, ranking and affordability.

For Proposition S.7, induct on the realized demand list. The induction step uses $c_w(K, A) + c_w(K \cup A, B) = c_w(K, A \cup B)$. Adding the recurring-work sum yields the complete accounting identity. Expanding both sides of Equation (S.20) and normalizing the conditional execution law proves the two-stage expression, with arbitrary dependence on the generated child.

For Proposition S.8, optimistic bounds give $v_t(a_t^*) - \gamma m_t(a_t^*) \le U_t(a_t^*) - \gamma L_t(a_t^*)$. Maximality bounds this by the chosen index. Subtracting the chosen true net value leaves at most $s_t + \gamma e_t$; finite summation proves the result. `DynamicCompletion` checks the identities, this bound, zero-weight padding, and the fresh-unit-resource sequence whose total bill is T. Its statements quantify over the realized cache state and do not require a predeclared finite universe of generated programs.

For checkpoint-conditioned predictions, finite summation preserves $0 \le \widehat{w}_{t,r} \le \bar{w}_r$. Averaging under any nonnegative continuation law preserves the same domination. Apply the shared-cache theorem with the fixed envelope $\bar{w}$: its coverage threshold and constants still apply, and coverage zeros every dominated live penalty. The declaration `checkpoint_conditioned_shared_root_log_bound` checks this complete composition, including history-dependent prices and continuation laws.

4 Formal Verification and Reproduction Map

The project pins Lean 4.24.0 and mathlib v4.24.0. Its unified entry is `theory/lean/AgentUCTPaper.lean`. The original operational development and archived conditional stochastic results remain compiled. `TreeGraphInformation`, `TreeFiniteBudget`, `TreeSharedCache`, and `TreeNextUnpaid` cover the compression witness, budget conditions, shared-panel rate, and first unpaid cost. The current revision also includes `TreeDemandQuotient`, `TreeDecisionValue`, `DynamicCompletion`, `ReachableDemandSeparation`, and `OnlineMean`. The integrated build, four language-server checks, and dependency audit pass; evidence is under `theory/evidence/reviewer-revision` and `theory/evidence/p0-p1`. The latter contains 17 added theorem checks and three completed language-server requests with zero diagnostics. The audited dependencies are Lean's standard `propext`, `Classical.choice`, and `Quot.sound`; there are no admitted proofs or project-defined axioms.

The main theorem quantifies its instance gap and constants before all finite event histories. It discharges child-deficit bounds by induction, without assuming a concentration event, logarithmic drift budget, or desired root-regret conclusion.

The online implementation mapping uses `OnlineMean`, whose envelope and sampled-law lemmas connect incremental means and finite completion samples to `online_mean_shared_root_log_bound`. The integrated build is recorded in `theory/evidence/algorithm/lean-build.log`; dependencies are listed in `theory/evidence/algorithm/lean-axioms.log`. The source declarations define the formal claims. Python refinement is not machine-verified. The historical event audits in `evidence/acquisition_update` and `evidence/theory_alignment` concern their recorded runtime revisions. Current runtime regressions and receipt evidence are under `evidence/main-pair-v1`; full reconstruction of the current experimental trajectories remains a separate post-run analysis. Paths are relative

to the workspace root.

The shared-cache theorem permits a fixed finite family of query panels with separate reward statistics. Its state-conditioned extension requires successful complete evaluations, persistent local cache entries, a fixed finite nonnegative prediction envelope, and deterministic reward tables. Arbitrarily changing the queries behind a shared reward mean, eviction, stochastic rewards, and action-dependent recurring penalties lie outside that theorem. The archived conditional stochastic results retain their stated assumptions.

Current proof inventory. This revision builds with 3305 Lake jobs, checks 96 declaration references in `AgentUCTProofGuideEN`, and audits all 385 public theorems in the 45 mathematical source files. Dependencies are `propext`, `Classical.choice` and `Quot.sound`; the source scan finds no admitted proofs or custom axioms. Logs and inventory are under `theory/evidence/proof-guide-en-20261004`. The English complete-proof file assembles the 45 modules; the original sources remain the editing locations.

4.1 Proven Results and Extension Directions

The proofs establish finite-tree quality rates under fixed rewards and persistent successful publication, exact execution accounting and full-panel cost quotients, and same-state pricing certificates with explicit error assumptions. Training, conditional validation and finite-history dynamic results retain their stated assumptions.

Extensions to stochastic rewards, publication failure or eviction require new operational models. Certified forecast errors and same-state costs of unselected candidates would connect the certificates to measured savings. Test generalization, growing-tree quality rates and Python refinement require separate distribution, comparator and transition models.

Equal-budget test F1 remains an empirical comparison. The rate counts completed evaluations, including free replays. A frozen empty-cache penalty need not vanish after coverage; boundedness alone does not transfer the theorem to that ablation. Extensions are assessed separately.

5 AI-Assisted Development

OpenAI Codex assisted with theory formulation, proof drafting and Lean implementation, experimental-protocol discussion, runtime code, saved-result analysis, simulated reviewer feedback, and manuscript revision. This use included methodological and experimental-design work. Retained development logs record model identifiers `gpt-6-astra` and `gpt-6.1-sol`, and Codex client versions 0.159.0, 0.159.2 and 0.160.0. These are recorded identifiers, not inferred model release numbers.

Selected original design prompts and English translations are supplied in `evidence/main-pair-v1/ai-assistance-disclosure-20261004.json`. They requested analysis of graph compression and the rationale for cost-aware allocation; construction of the full-evaluation cost estimator; treatment of cache accumulation during rollout; actual-cost snapshot reporting; uniform batch execution; and validation-selected test comparisons across budgets. The inventory covers 15 retained development sessions and identifies the observed model set at session level. It is not a complete transcript or an inventory of unrecorded interactions.

The subsequent manuscript restructuring used the user-requested `nature-writing` 1.5.0 and `nature-figure` 2.8.0 guidance from https://github.com/Yuan1z0825/nature-skills. The method schematic was programmatically drawn with Matplotlib; it contains illustrative symbols, not generated experimental observations or image-model artwork. The disclosure file records the additional budget-analysis, token/cache reporting, paired statistical analysis, configuration-table and visual-reference requests. Supplementary prose also used `write-like-me` 0.1.1 for the author's requested direct style.

The delivered Lean sources and compiler logs support formal statements. Saved evaluations and receipts support empirical claims. The authors remain responsible for accuracy, citations and disclosure.